

Knowledge, Data and Code

Lecture 02 | Purpose, distinctions and modelling consequences

Data → Information → Knowledge → Code
Declarative vs procedural representation

Learning objectives

- Distinguish data, information, knowledge and code by their purpose in an intelligent system.
- Explain declarative and procedural representations.
- Compare a knowledge base with a conventional program.
- Explain why KRR separates domain knowledge from inference mechanisms.
- Analyse the modelling consequences of putting a fact or policy in data, knowledge or code.

A simple question: where should this go?

Observation

Motor temperature = 92°C

Domain knowledge

Above 90°C while running means overheating.

Behaviour

If overheating, create an alert and recommend inspection.

All three can be written in software, but they play very different roles. Good modelling starts by separating those roles.

Data: recorded observations

- Data is a recorded value, symbol, event or measurement.
- By itself, data may have little meaning outside its context.
- Examples: 92, M7, 14:32, ON, vibration = 7.4 mm/s.
- Data usually changes frequently and is often stored in files, tables, streams or databases.

Typical question

What has been observed or recorded?

Information: data interpreted in context

DATA

Raw data

92
M7
14:32
RUNNING

Information

At 14:32, Machine M7 was RUNNING and its motor temperature was 92°C.

Context supplies labels, units, identity, time and relationships. That turns isolated data into something meaningful to a user or system.

Knowledge: meaning that supports reasoning

- Knowledge captures facts, categories, relationships, rules, constraints and assumptions about a domain.
- It is not merely a stored value. It tells the system how concepts relate and what follows from them.
- Example: A running motor above 90°C is considered overheated.
- From information + knowledge, the system can derive: M7 is overheating.

Code: instructions for computation

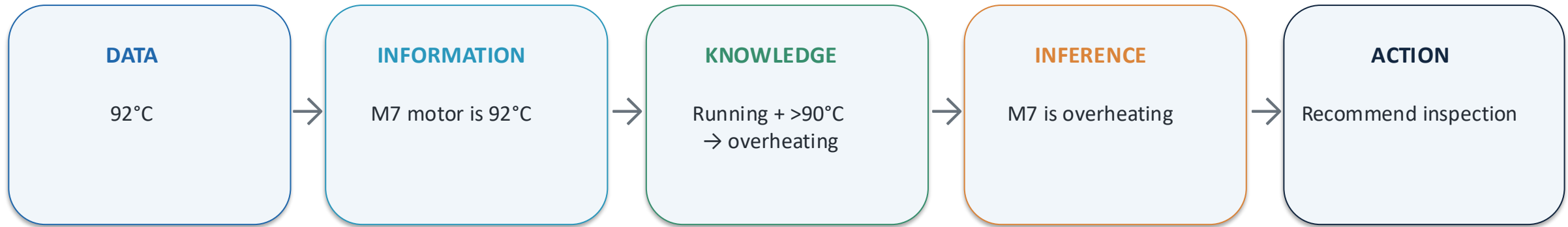
- Code specifies operations the computer should execute.
- It may implement algorithms, workflows, interfaces, storage access and inference procedures.
- Domain knowledge can also be hard-coded inside program logic, but then knowledge and procedure become tightly coupled.
- Code answers primarily: How should the system perform a computation?

Example

```
if temperature > 90 and running: status = 'OVERHEATING'
```

Data → information → knowledge → action

DISTINCTIONS



Code provides the machinery that stores, transforms, reasons over and acts on these representations.

The same domain can be encoded in different places

As data

threshold = 90

Easy to change as a configuration value, but meaning may be weak unless the schema explains it.

As knowledge

$\text{HighTemp}(x) \wedge \text{Running}(x) \rightarrow \text{Overheating}(x)$

Explicit, queryable and explainable.

As code

if temp > 90 and running:
 overheating = True

Executable, but the domain rule is embedded in procedure.

The modelling decision affects maintainability, explainability, reuse and reasoning capability.

Declarative representation

- Declarative representation states what is true, permitted, required or related.
- It focuses on domain knowledge rather than the exact control flow used to process it.
- Examples: facts, logical formulae, rules, ontologies, constraints and knowledge-graph triples.
- The same knowledge can often be used by multiple queries or reasoning tasks.

Example

`critical(Pump4). highVibration(Pump4). critical(x)  $\wedge$  highVibration(x)  $\rightarrow$  inspect(x)`

Procedural representation

- Procedural representation specifies how to achieve a result through a sequence of operations.
- Control flow is explicit: conditions, loops, function calls and state changes.
- Knowledge about the domain may be distributed across many procedures.
- Procedures are often efficient for a fixed task but can be harder to inspect as a coherent knowledge model.

Example

```
if asset.critical and asset.vibration > limit: create_inspection(asset)
```

Declarative vs procedural: what changes?

Declarative

Emphasis: WHAT is known

- Knowledge is explicit
- Easier to query and explain
- Often easier to modify domain rules
- Inference engine determines how conclusions are found
- Supports reuse across tasks

Procedural

Emphasis: HOW to compute

- Control flow is explicit
- Often straightforward to execute
- Domain knowledge may be mixed with implementation
- Changes can require code modification
- Behaviour is tied to procedures

Knowledge base vs conventional program

Knowledge-based system

Knowledge Base

Facts • rules • ontology • constraints

+

Inference / reasoning mechanism

+

Queries or goals

Conventional program

Input

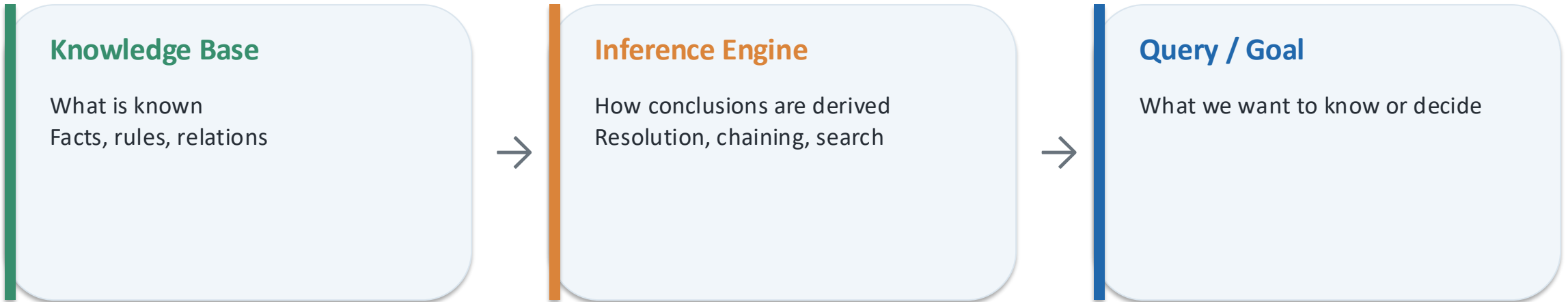
↓

Program logic containing operations and domain decisions

↓

Output

Separate knowledge from inference



This separation is one of the central ideas behind knowledge-based systems and the original KRR lecture's contrast between declarative knowledge and procedural behaviour.

Why separation matters

- Modify domain knowledge without redesigning the whole reasoning algorithm.
- Reuse one inference mechanism with different knowledge bases.
- Reuse one knowledge base for multiple questions and tasks.
- Inspect which facts and rules produced a conclusion.
- Test knowledge separately from software infrastructure.
- Allow domain experts to review knowledge in a form closer to the domain.

Modelling consequence 1: maintainability

CONSEQUENCES

Hard-coded policy

A safety threshold appears in 12 functions.

Changing the policy requires finding and updating every implementation point.

Risk: inconsistent behaviour.

Represented policy

The threshold or rule is maintained once in a configuration/knowledge layer.

Multiple services use the same definition.

Benefit: controlled change.

Modelling consequence 2: explainability

Opaque procedural result

“The system returned HIGH_RISK because branch 47 evaluated to true.”

Knowledge-level explanation

“Pump-4 is critical; vibration is above the defined limit; the maintenance rule therefore recommends inspection.”

Explicit knowledge gives us vocabulary for explaining decisions in domain terms rather than only implementation terms.

Modelling consequence 3: consistency and reuse

- If the same concept is encoded differently in databases, rules and code, contradictions can appear.
- A shared vocabulary or ontology can make meanings explicit across components.
- Rules can then refer to stable concepts rather than application-specific field names.
- Reuse improves when knowledge is represented independently of one particular screen, API or workflow.

Practical AI example: customer support agent

EXAMPLES

Data

Customer ID
Order status
Message text
Timestamps

Knowledge

Refund policy
Product hierarchy
Escalation rules
Known issues

Code

API calls
Authentication
UI
Retrieval logic

Reasoning

Is refund allowed?
Which rule applies?
Should the case escalate?

Practical AI example: industrial maintenance agent

EXAMPLES

Operational data

Temperature
Vibration
Run state
Alarm history

Domain knowledge

Asset hierarchy
Failure modes
Thresholds
Maintenance rules

Software / inference

Stream processing
Rule evaluation
Queries
Notification actions

A strong design does not treat the entire system as one large block of application code.

Practical AI example: LLM + knowledge layer

- An LLM contains learned statistical patterns, but an application often also needs explicit enterprise knowledge.
- Data may come from documents, databases, sensors or user interactions.
- A knowledge layer can represent policies, entities, relations, constraints and provenance.
- Code connects the model to retrieval, tools and external systems.
- Reasoning or rule checks can constrain or validate generated recommendations.

Design lesson

Do not ask one component to play every role.

A useful modelling checklist

- ✓ Is this a changing observation? → probably data.
- ✓ Is this a stable domain fact, relationship, policy or constraint? → consider explicit knowledge.
- ✓ Is this an algorithm or operation? → code.
- ✓ Must the system explain or query the rule itself? → avoid hiding it only in code.
- ✓ Will the same knowledge be reused by several tasks? → separate it from task-specific procedures.
- ✓ Does the representation preserve enough meaning for correct reasoning?

Lecture 02 takeaways

- Data records observations; information gives them context; knowledge captures meaning that supports inference; code specifies computation.
- Declarative representation emphasizes what is known. Procedural representation emphasizes how a result is produced.
- A knowledge-based system separates domain knowledge from inference mechanisms.
- Where knowledge is represented has real consequences for change, reuse, consistency, testing and explanation.
- Good KRR modelling makes important domain assumptions explicit rather than burying them inside application code.

Reading and connection to the course

- Brachman & Levesque, Knowledge Representation and Reasoning: introductory perspective on representation and reasoning.
- Russell & Norvig, Artificial Intelligence: A Modern Approach: knowledge-based agents and logical representation.
- Next: Propositional Logic — syntax, semantics, entailment and models.